

РОБОПОДБОР — сопроводительная документация

Веб-платформа экспресс-предынвестиционной оценки роботизации: каталог решений, подбор под объект, расчёт экономики, сравнение сценариев, имитация работы роботов и выгрузка отчёта.

Быстрый старт

```
docker compose up --build
```

Откройте <http://localhost:8080>. Демонстрационные учётные записи:

Роль	Логин	Пароль
Пользователь	krylov@volga-logistic.ru	volga123
Администратор	admin@robotopodbor.ru	admin123

Гостю вход не нужен: каталог и расчёт доступны сразу. Сайт на GitHub Pages работает без сервера, на встроенных моках, с теми же учётными записями.

Состав документации

Пункт ТЗ	Документ
6.1 Целевая аудитория, сценарии, границы решения	01-product.md
6.2, 6.10 Развёртывание, запуск, воспроизведение демо-расчёта	02-deploy.md
6.3 Архитектура, компоненты, структура данных, API	03-architecture.md
6.4 Алгоритм подбора и ранжирования	04-selection.md
6.5 Экономическая модель	05-economics.md
6.6 Загрузка и актуализация данных	06-data.md
6.7 Руководство пользователя	07-user-guide.md
6.7 Руководство администратора	08-admin-guide.md
6.8 Внешние библиотеки и источники данных	09-dependencies.md
6.9 Ограничения и план развития	10-limitations.md
3.8.3 Возможные интеграции	11-integrations.md

Пример выгруженного отчёта (ТЗ 8.2.7) — [examples/primer-otcheta-sklad.pdf](#) и [.xlsx](#): склад из датасета организатора, погрузчик АК-2000-2.

Спецификация API — [contracts/openapi.yaml](#) (OpenAPI 3.1).

1. Целевая аудитория, сценарии и границы решения (ТЗ 6.1)

Для кого

- Руководители предприятий и подразделений, которым нужно обосновать бюджет на роботизацию перед инвестиционным комитетом.
- Технические директора, руководители автоматизации и цифровой трансформации.
- Логисты, операционные и финансовые аналитики, готовящие предварительное обоснование.
- Дополнительно — интеграторы и поставщики, которые собирают расчёт для своих клиентов.

Платформа независима от производителей: она не продаёт технику и не получает комиссию за внедрение.

Пользовательские сценарии

Сценарий ТЗ	Как реализован
2.1.1 Гость смотрит каталог и делает демо-расчёт	<code>/catalog</code> , <code>/catalog/compare</code> , мастер <code>/calculate/</code> : тип без входа. Незаполненные поля паспорта берутся из датасета организатора, поэтому расчёт работает без ввода данных
2.1.2 Пользователь создаёт проект, вводит или загружает параметры, получает подборку	Паспорт объекта вводится вручную или загружается из Excel/CSV по шаблону. Подбор показывает рекомендованные решения, решения «требуется проверки» и исключённые с причинами
2.1.3 Сравнение базового сценария и сценариев покупки и услуги	Страница результатов: сценарии, CAPEX/OPEX, окупаемость, ROI, риски
2.1.4 Имитация работы решения	3D-симуляция склада на странице результатов по параметрам расчёта и характеристикам выбранного робота
2.1.5 Сохранение и выгрузка	Раздел «Расчёты»: сохранение, копирование, переименование, история расчётов. Отчёт в PDF и расчётные таблицы в Excel
2.1.6 Администратор актуализирует данные	<code>/admin</code> : источники данных, нормативы и значения по умолчанию, журнал изменений. <code>/admin/catalog</code> : каталог, импорт и экспорт таблицы решений

Роли: гость, пользователь, администратор (ТЗ 3.1.1). Роль вендора не реализована.

Типы объектов

Склад, аэропорт и медицинское учреждение. Для каждого типа есть свой паспорт с обязательными и дополнительными полями (склад — 46 полей, аэропорт — 39, медучреждение — 57) и демонстрационный набор значений. Новый тип объекта добавляется данными (запись `ObjectTypeDef` и её поля `ParameterField`), без изменения кода ядра.

Границы решения

- Это экспресс-оценка, а не проектирование. Результат — гипотеза о целесообразности роботизации и материал для перехода к ТЭО. Каждый отчёт явно помечен как предварительная оценка, требующая верификации при обследовании объекта (ТЗ 3.7.5).
- Интеграции с WMS, ERP, 1C и государственными сервисами не реализованы. Возможные варианты описаны в [11-integrations.md](#).
- Все данные демонстрационного набора обезличены. Реальные персональные данные не используются (ТЗ 4.4.5).
- Демонстрация не зависит от внешних сервисов: каталог, паспорта и демо-расчёт загружаются в базу при первом запуске (ТЗ 4.2.7).

2. Развёртывание и запуск (ТЗ 6.2, 6.10)

Требования

- Docker 24+ с Docker Compose v2.
- 2 ГБ свободной памяти, порт 8080.
- Интернет нужен только при сборке образов (пкм-пакеты). Во время работы внешние сервисы не нужны.

Пользователю нужен только современный браузер. Специального оборудования и видеоускорителя не требуется.

Запуск одной командой

```
git clone <репозиторий> robopodbor && cd robopodbor
docker compose up --build
```

Что происходит:

1. `db` — PostgreSQL 16, данные хранятся в томе `db-data`.
2. `migrate` — применяет миграции Prisma и заполняет пустую базу: типы объектов, поля паспортов, каталог решений, источники данных, демонстрационные учётные записи. Повторный запуск данные не дублирует.
3. `api` — сервер Express на порту 4000 внутри сети compose. Наружу он не публикуется.
4. `web` — nginx на порту **8080**: раздаёт интерфейс и проксирует `/api` на сервер. Ограничивает частоту запросов: 10 в минуту на вход, 30 в секунду на остальной API.

Проверка готовности: `curl http://localhost:8080/api/health` должен вернуть `{"status": "ok"}`.

Остановка — `docker compose down`. Полный сброс вместе с базой — `docker compose down -v`.

Настройки

Без файла `.env` поднимаются демонстрационные значения — для локального запуска он не нужен. Чтобы сменить порт или разместить сервис в сети, скопируйте `.env.example` в `.env` и раскомментируйте нужные строки:

Переменная	Назначение
<code>WEB_PORT</code>	порт сайта на хосте, если 8080 занят (по умолчанию 8080)
<code>POSTGRES_PASSWORD</code>	пароль базы данных
<code>WEB_ORIGIN</code>	адрес, с которого открывают интерфейс (для CORS)
<code>DOMAIN</code>	домен для HTTPS-сертификата (профиль <code>tls</code>)
<code>WEB_BIND</code>	интерфейс, на котором слушает nginx; с TLS — <code>127.0.0.1</code>
<code>TRUST_PROXY_HOPS</code>	число прокси перед API: 1 без TLS, 2 с Caddy

HTTPS (ТЗ 4.4.4)

```
docker compose --profile tls up -d --build
```

Caddy получает сертификат Let's Encrypt для `DOMAIN` и включает HTTP/2 и HTTP/3. Настройки — в `deploy/Caddyfile`.

Сайт на GitHub Pages с настоящим сервером

По умолчанию сайт на GitHub Pages работает на встроенных моках: «база» (аккаунты, проекты, расчёты) хранится в `localStorage` браузера. Чтобы сайт работал с настоящим API:

1. Поднимите стек на сервере с доменом: `docker compose --profile tls up -d --build`. В `.env` укажите `WEB_ORIGIN=https://<user>.github.io` и `CROSS_SITE_COOKIES=true`: сайт и API окажутся на разных доменах, и cookie сессии нужен `SameSite=None; Secure`.
2. В репозитории, в Settings → Secrets and variables → Actions → Variables, задайте `API_URL=https://<DOMAIN>/api`. Следующая публикация соберёт сайт с `VITE_ENABLE MOCKS=false` и адресом API.

Запуск для разработки

Нужен Node.js 22.

```
# API: база из compose, сервер – локально
docker compose up -d db
cd apps/api
printf
'DATABASE_URL=postgresql://robopodbor:robopodbor@localhost:5432/robopodbor\nWEB_ORIGIN=http://localhost:5173\nPORT=4000\n'
> .env
npm ci
npx prisma migrate deploy
npx tsx --env-file=.env prisma/seed.ts
npm run dev                # http://localhost:4000

# Интерфейс
cd apps/web
npm ci
npm run dev                # с моками MSW, без сервера
VITE_ENABLE MOCKS=false npm run dev    # с настоящим API через прокси /api
```

Порт 5432 наружу не публикуется. Для локальной разработки добавьте `ports: ["5432:5432"]` сервису `db` в `docker-compose.override.yml`.

Проверки: `npm run typecheck`, `npm run lint`, `npm test` (в `apps/web`), `npm run build` (в обоих приложениях).

Воспроизведение демонстрационного расчёта (ТЗ 6.10)

1. Откройте <http://localhost:8080>, нажмите «Рассчитать» и выберите «Склад».
2. На шаге паспорта нажмите «Скачать шаблон Excel» — это пример файла. Можно ничего не вводить: незаполненные поля берутся из демо-набора, у каждого поля указано значение по умолчанию и его источник.
3. На шаге процессов посмотрите подбор: рекомендованные решения, «требуется проверки» и свёрнутый список «Не подходят для объекта» с причинами.
4. Выберите решение и запустите расчёт. На странице результатов — KPI, сценарии «без роботизации», «покупка» и «услуга (RaaS)», структура затрат, чувствительность и визуализация.
5. Выгрузите PDF и Excel кнопками экспорта.
6. Войдите как `krylov@volga-logistic.ru / volga123`, пройдите мастер и в разделе «Расчёты» нажмите «Сохранить черновик». В меню строки проекта откройте «История расчётов»: у снимка указаны дата, версия данных и версия модели. Кнопка «Открыть» воспроизводит сохранённый результат.
7. Войдите как `admin@robotopodbor.ru / admin123` и откройте «Справочники» в боковом меню, чтобы посмотреть источники, нормативы и журнал изменений.

3. Архитектура, компоненты, данные и API (ТЗ 6.3)

Компоненты



Компонент	Технологии	Путь
Интерфейс	React 19, Vite, TanStack Query, Zustand, Tailwind, Radix UI, Recharts, three.js	apps/web
API	Node.js 22, Express 4, Prisma 6, zod, multer, SheetJS	apps/api
Общие доменные правила	TypeScript без зависимостей: каталог, подбор, расчёт экономики, паспорта, табличный импорт, версии	apps/api/src/domain
Контракт	OpenAPI 3.1, типы клиента генерируются из него	contracts/openapi.yaml
СУБД	PostgreSQL 16	сервис db
Прокси	nginx: сжатие, кеш статика, заголовки безопасности, ограничение частоты запросов	apps/web/nginx.conf

Правила подбора, расчёт экономики (`economics.ts`), проверки паспорта и описание каталога лежат в `apps/api/src/domain` и подключаются в интерфейс через псевдоним `@domain`. Поэтому сервер, демо-режим с моками MSW и подсказки в формах дают одинаковый результат.

Расширяемость (ТЗ 4.2.6)

- **Новый тип объекта** — запись `ObjectTypeDef` и набор `ParameterField` в данных. Формы, шаблон Excel, импорт и проверка диапазонов строятся по этим записям.
- **Новый продукт или тип решения** — через админку, импорт таблицы или API каталога. Иерархия «отрасль → объект → процесс → тип решения → продукт» строится из полей решения (`buildTaxonomy`).
- **Новое правило подбора** — функция-проверка в `domain/selection.ts`. Версия правил (`SELECTION_RULES_VERSION`) возвращается вместе с подбором.
- **Расчётный модуль** — `domain/economics.ts`, результат по контракту `CalculationResult`. `POST /calculations` считает и сохраняет расчёт мастера, сохранение в проект — `POST /projects/{id}/calculations` с указанием версии модели. Новый тип объекта получает свои замещающие роли и потоки в таблице профилей процессов этого модуля.

Безопасность (ТЗ 4.4)

- Сессия хранится в `httpOnly-cookie` `sid`, запись сессии лежит в базе и имеет срок жизни.
- Пароли хранятся как соль + `scrypt`, открытого вида нет.
- Доступ по ролям: `middleware requireAuth` и `requireRole("admin")`. Проекты выбираются только по `userId` владельца (изоляция, 4.4.3).
- Удаление проекта каскадно удаляет его сценарии и историю расчётов. Загруженные файлы разбираются в памяти и на диск не пишутся (4.4.6).
- Заголовки безопасности (`helmet` и `nginx`), ограничение частоты запросов, HTTPS через Caddy.
- Из загружаемых `xlsx` читаются только значения ячеек: формулы и макросы не исполняются. Размер файла — до 5 МБ.

Модель данных

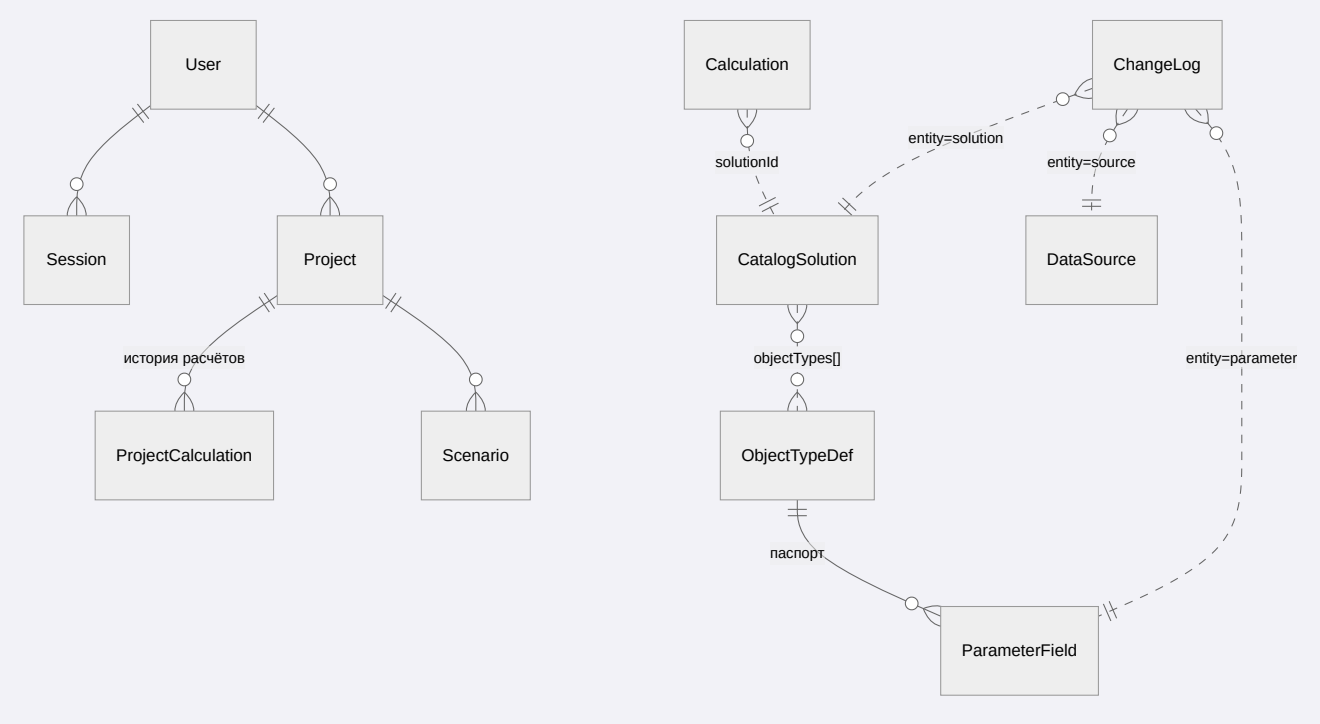


Таблица	Назначение
User, Session	Учётные записи (роль user или admin), избранные роботы (favoriteSolutionIds), дата смены пароля и сессии
Project	Проект: тип объекта, значения паспорта, выбранные процессы и решение, последний снимок расчёта с dataVersion, modelVersion, calculatedAt
ProjectCalculation	История расчётов: каждый сохранённый результат вместе с паспортом, версией данных и версией модели (ТЗ 3.1.5)
Scenario	Сценарии проекта для списка и дашборда
Calculation	Расчёт мастера: входы (тип объекта, паспорт, процессы, решение), результат, версии данных и модели. Гостевые расчёты удаляются через 90 дней
ObjectTypeDef	Тип объекта: склад, аэропорт, медучреждение
ParameterField	Поле паспорта: подпись, единица, раздел, тип, диапазон, значение по умолчанию, обязательность, источник норматива
CatalogSolution	Решение каталога. Группы полей повторяют таблицу 3.3 ТЗ: идентификация, технические характеристики, ограничения для подбора, инфраструктура, экономика, применимость, качество данных
DataSource	Источник данных: тип, область, ссылка, дата актуализации, признак подтверждённости
ChangeLog	Журнал изменений справочников: кто, когда, что, до и после

Версия данных — отметка времени последней записи в ChangeLog (или data-seed для исходного набора). **Версия модели** — идентификатор расчётной модели, выдавшей результат. Обе версии сохраняются в каждом снимке расчёта. Если справочники менялись после расчёта, это видно по версии, а сохранённый результат при повторном открытии не пересчитывается.

Схема базы — apps/api/prisma/schema.prisma, миграции — apps/api/prisma/migrations.

API (ТЗ 3.8.2, 3.8.5)

Все пути начинаются с /api. Полное описание запросов, ответов и ошибок — в contracts/openapi.yaml. Контракт проходит проверку Redocly, и каждому маршруту сервера соответствует операция в контракте.

Группа	Методы	Доступ
Здоровье	GET /health	все
Вход	POST /auth/register, /auth/login, /auth/logout, GET /auth/session	все
Личный кабинет	PATCH /auth/profile — имя и организация; POST /auth/password — смена пароля (другие сессии закрываются); GET /auth/favorites, PUT, DELETE /auth/favorites/{solutionId} — избранные роботы	вошедший пользователь
Типы объектов и паспорта	GET /object-types, /object-types/{slug}/parameters, /object-types/{slug}/parameters/template; POST /object-types/{slug}/parameters/import	все
Каталог	GET /catalog/solutions, /catalog/taxonomy	все

Группа	Методы	Доступ
Каталог, изменение	POST, PUT, DELETE /catalog/solutions[/{id}]; GET /catalog/solutions/export; POST /catalog/solutions/import	админ
Подбор	POST /selection	все
Расчёт	POST /calculations — посчитать и сохранить; GET /calculations/{id} — расчёт мастера, проект, снимок из истории или demo	все; расчёт вошедшего пользователя и проекты — только владельцу
Проекты	GET, POST /projects; GET, PATCH, DELETE /projects/{id}; POST /projects/{id}/copy; GET, POST /projects/{id}/calculations	только владелец
Источники	GET /sources; POST, PUT, DELETE /admin/sources[/{id}]	чтение — все, запись — админ
Нормативы	PATCH /admin/parameters/{fieldId}	админ
Журнал	GET /admin/changes?entity=&limit=	админ

Типы клиента пересобираются из контракта: `cd apps/web && npm run gen:api`.

Производительность (ТЗ 4.3)

- Справочники (типы объектов, паспорта, каталог, дерево каталога) кешируются в памяти API на 60 секунд и сбрасываются сразу при правке. Браузер получает их с заголовками кеширования.
- Индексы: проекты по владельцу и дате, история по проекту и дате, GIN-индекс по типам объектов каталога.
- Тяжёлые части интерфейса (3D, графики, экспорт) загружаются отдельными чанками по требованию.

Нагрузочный тест (ТЗ 4.3.1)

50 пользователей одновременно и без пауз 60 секунд проходят сценарий гостя: типы объектов → паспорт → каталог → подбор → расчёт → открытие отчёта. Скрипт — `scripts/load/scenario.mjs`, запуск: `node scripts/load/scenario.mjs`
`http://localhost:4000/api 50 60`.

Прогон 28.09.2026: один экземпляр API в режиме разработки (tsx), PostgreSQL 16 на той же машине, Apple M1, 8 ядер, без nginx.

Запрос	Запросов	p50, мс	p95, мс	p99, мс	Ошибок
GET /object-types	5 687	43	114	233	0
GET /object-types/{slug}/parameters	5 687	44	145	841	0
GET /catalog/solutions?objectType=	5 687	56	174	413	0
POST /selection	5 687	65	182	442	0
POST /calculations	5 687	144	310	903	0
GET /calculations/{id}	5 687	67	169	335	0

Всего 34 122 запроса, 561 в секунду, ошибок нет, самый долгий ответ — 1,5 с. Нагрузка выше реальной: пользователи не делают пауз между шагами.

Режим без сервера (GitHub Pages)

Если сайт собран без `VITE_ENABLE MOCKS=false`, запросы `/api` обрабатывает MSW в браузере по тому же контракту и с теми же доменными модулями. Аккаунты, сессии, проекты и расчёты хранятся в `localStorage` (`apps/web/src/mocks/persist.ts`) и переживают перезагрузку. С переменной `VITE_API_URL` сайт работает с настоящим API на другом домене, см. [02-deploy.md](#).

4. Алгоритм подбора и ранжирования (ТЗ 6.4, 3.4)

Код — `apps/api/src/domain/selection.ts`, версия правил `selection-2026.09d`. Вызов — `POST /api/selection` с телом `{ objectType, parameters }`. Тот же модуль работает в интерфейсе, поэтому результат на сервере и в демо-режиме совпадает.

Шаги

- Паспорт объекта.** Незаполненные поля берутся из значений по умолчанию паспорта. Поэтому подбор работает и в гостевом демо без ввода данных (ТЗ 2.1.1).
- Кандидаты.** В подбор попадают решения каталога, у которых в `objectTypes` есть выбранный тип объекта (3.4.1). Каталог — реальный каталог роботов (`apps/api/prisma/seed-data/robots-catalog`), его переводит в решения скрипт `build-catalog.ts`. Тип объекта попадает в `objectTypes` двумя путями, это записано в `objectFit`:

- `declared` — объект указан в каталоге (поле `site`);
- `example` — организатор приводит решение как пример для этого объекта («Примеры решений», `seed-data/catalog-supplements.json`); считается подтверждённым, как `declared`;
- `inferred` — объект не указан, но сценарий применения и категория робота подходят объекту. Правила — `FIT_RULES` в `src/domain/robotCatalog.ts`. Сверх правил:
 - складские AMR и погрузчики (категории AM, FL) подходят и грузовому терминалу аэропорта — организатор приводит их как пример для аэропорта;
 - `FIT_OVERRIDES` — решения по конкретным роботам с обоснованием в коде (разбор 28.09.2026): AMR цеховой логистики для склада, небольшие AMR и тягач для клиники, уличные уборщики для территории аэропорта, БАС мониторинга для инспекции аэропорта и охраны склада, БАС доставки биоматериалов для клиники; по решению заказчика — Protonobot, робо-кафе и беспилотный автобус («Обслуживание пассажиров»), пожарный лафет («Пожарная безопасность») для аэропорта, Яндекс-ровер для клиники. Коботы и ячейки укладки на склад не включены.

Из того же файла дополнений генератор заполняет пустые характеристики каталога и добавляет решения, которых в каталоге нет (PuduBot 2 — EX0001). Заполненные значения каталога не перезаписываются: расхождения генератор печатает для ответственного за каталог.

- Проверки.** Каждое ограничение продукта сравнивается с параметром объекта. Итог проверки попадает в один из четырёх списков:

- причины соответствия** — ограничение проверено и выполняется;
- блокеры** — ограничение делает применение невозможным;
- ограничения** — применять можно, но есть условия;
- недостающие данные** — проверить нельзя.

4. Статус.

- `excluded` («Не подходит») — есть хотя бы один блокер. Такое решение не рекомендуется, его балл равен 0 (3.4.3).
- `needs-review` («Требует проверки») — блокеров нет, но хотя бы одну проверку «подходит / не подходит» выполнить не удалось, применимость к объекту выведена по сценариям (`inferred`), характеристики не подтверждены или решение ещё не в серийной эксплуатации (пилот, НИОКР).
- `recommended` («Рекомендовано») — все проверки выполнены, решение в эксплуатации.

- Сортировка.** Сначала рекомендованные, затем требующие проверки, затем исключённые. Внутри группы — по баллу.

Проверки

Проверка	Параметр объекта	Характеристика решения	Блокер, если
Тип объекта	выбранный тип	<code>objectTypes</code>	тип не поддерживается
Грузоподъёмность	масса паллеты или штучной единицы (склад), масса багажа — только для роботов багажа и перрона (аэропорт), масса тележки с питанием — только для процесса «белье и питание» (медучреждение)	<code>payloadKg</code>	грузоподъёмность меньше массы. Складской мобильный робот, который не берёт паллету, но берёт штучный груз, не исключается — получает ограничение «только лёгкий груз»
Ширина прохода (мобильные решения)	ширина проходов (склад), ширина коридоров (медучреждение)	<code>minAisleWidthM</code>	проход уже требуемого
Ровность пола	ровность пола, мм на 2 м	<code>maxFloorDeviationMm</code>	пол хуже допуска
Высота помещения	высота потолков в зоне хранения	<code>minCeilingHeightM</code>	потолки ниже

Проверка	Параметр объекта	Характеристика решения	Блокер, если
Температура (аэропорт, уличные решения)	температура в неотапливаемых зонах	minTempC	объект холоднее нижней рабочей температуры
Допуск на перрон (аэропорт)	требуемая сертификация	airsideCertified	допуска нет
Этажность (медучреждение, мобильные решения)	число этажей, лифты с API, пандусы	elevatorIntegration	этажей больше одного, интеграции с лифтами нет и пандусов нет
Бюджет	планируемый бюджет на роботизацию	цена единицы	цена одной единицы выше бюджета

Ограничения без исключения:

- шум выше нормы для зоны;
- у объекта нет WMS, а роботам она нужна;
- нужна интеграция со СКУД;
- лифты без API;
- ограничения из карточки решения.

Недостающие данные, при которых решение получает статус «Требуется проверки»:

- для проверки нет характеристики в карточке решения (например, ширины прохода);
- параметр не заполнен в паспорте объекта;
- производитель не указывал этот тип объекта — применимость выведена по сценариям;
- решение на стадии НИОКР;
- характеристики не подтверждены поставщиком.

Показываются, но статус не понижают:

- незаполненные обязательные поля карточки — в реальном каталоге заполнена малая часть полей, и иначе «Требуется проверки» получили бы все решения;
- допущения по полям карточки (например, страна происхождения взята по стране производителя).

Балл ранжирования (3.4.5)

Балл — сумма пяти критериев, максимум 100. В карточке решения показан вклад каждого критерия в виде «набрано / максимум».

Критерий	Максимум	Расчёт
Зрелость решения	30	эксплуатация — 30, пилот — 18, НИОКР — 6
Соответствие объекту	25	25 – 5 × число ограничений, не меньше 10
Цена относительно бюджета	20	20 × (1 – цена единицы / бюджет), в пределах 0...20; без бюджета — 10
Полнота и подтверждённость данных	15	15 × полнота карточки × (1, если данные подтверждены, иначе 0,7)
Российский производитель	10	Россия — 10, иначе 4

Веса — экспертное допущение команды. Они собраны в одном месте кода и меняются вместе с версией правил.

Ручное добавление (3.4.4)

Пользователь может выбрать исключённое решение вручную: на шаге процессов и в сравнении появляется предупреждение с причиной исключения. Выбор сохраняется, расчёт выполняется.

5. Экономическая модель (ТЗ 6.5, 3.5)

Расчётный модуль ведёт отдельная часть команды. Здесь описаны состав показателей, базовые формулы по ТЗ, принятые допущения и то, как результат модели попадает в платформу. Точные формулы и коэффициенты текущей версии модели (`modelVersion`) — в документации модуля и в выгружаемом отчёте.

Показатели (3.5.2)

Показатель	Базовая логика	Единица
Количество роботов	пиковая потребность в операциях / (производительность одного робота × коэффициент загрузки × доступность), плюс резерв	шт.
CAPEX	оборудование + инфраструктура + ПО + интеграция + пусконаладка + обучение + резерв	млн ₽
Годовой OPEX	сервис + лицензии + электроэнергия + связь + расходные материалы + ремонт + персонал эксплуатации	млн ₽/год
Изменение OPEX	OPEX сценария – OPEX базового сценария, в том числе экономия фонда оплаты труда	млн ₽/год
Чистый годовой эффект	сокращение текущих затрат + дополнительный доход или предотвращённые потери – дополнительный OPEX	млн ₽/год
Срок окупаемости	CAPEX / годовой эффект, если эффект положительный	лет
ROI	накопленный эффект за период / CAPEX × 100%	%
TCO	CAPEX + сумма OPEX за горизонт с учётом замены основных компонентов	млн ₽

Сценарии (3.5.5)

- **Как есть** — текущий процесс без роботизации.
- **Покупка** — CAPEX и собственная эксплуатация.
- **Услуга (RaaS)** — аренда: CAPEX минимальный, в OPEX — абонентская плата.

Для каждого сценария показывается чувствительность к стоимости оборудования, объёму операций и стоимости труда (3.5.6). Рекомендация строится не по жёсткому порогу, а по интервалам окупаемости (до 3 лет, 3–5 лет, больше 5 лет) с перечнем рисков (3.5.7).

Допущения по умолчанию

Допущение	Значение	Статус
Дисконтирование	не применяется: окупаемость по накопленному потоку	упрощение экспресс-оценки
Рост зарплат	9% в год	подтверждено
Горизонт расчёта	из паспорта объекта (датасет: склад 5 лет, аэропорт и клиника 7)	подтверждено
Замещение	1 робот = 1 пост в каждой смене	допущение команды
Внедрение, интеграция, зарядка	15% от цены оборудования, если в карточке нет своей цифры	оценка
Обслуживание техники	7% от цены в год, если в карточке нет своей цифры	оценка по данным вендоров
Тариф аренды (RaaS)	2,5% цены робота в месяц, сервис включён	оценка рынка
Электроснабжение	7 кВт·ч, один тариф для всех регионов	оценка
Операторы парка	1 на 10 роботов в смену	оценка

Пользователь может менять ключевые допущения на странице результатов: стоимость персонала, режим работы, производительность, стоимость оборудования и обслуживания, коэффициент загрузки, горизонт (3.5.3).

Как считается (модель `economics-2026.09b`, `apps/api/src/domain/economics.ts`)

Мастер вызывает `POST /api/calculations` с паспортом объекта, процессами и выбранным решением. Тот же модуль работает и на сервере, и в моках сайта. Результат сохраняется и открывается по ссылке `/results/{id}`.

1. **Замещаемый труд.** По процессу решения выбираются роли из паспорта: для склада — операторы погрузчиков (транспорт, приёмка, хранение) или отборщики (отбор, сортировка); для аэропорта — наземный персонал (багаж, перрон, грузы) или операторы уборочных машин; для клиники — санитары-транспортировщики (доставка) или сотрудники прачечной и пищеблока

(бельё и питание). ФОТ = численность × з/п × 12 × коэффициент начислений. Для уборки склада, охраны и инвентаризации в паспорте нет персонала, поэтому окупаемость не считается, а в отчёте прямо написано, каких данных нет.

2. **Число роботов.** Пиковый поток / (производительность робота × 0,85 × 0,95) + 5% резерва. Производительность берётся из карточки. Для погрузчиков и AMR без производительности она считается по скорости и маршруту (половина стороны склада туда и обратно + 60 с на погрузку). Если производительность взять неоткуда, закладывается 1 робот на 1 пост в смене, но не больше, чем позволяет бюджет на роботизацию из паспорта.
3. **Сколько людей замещено.** 1 робот = 1 пост в каждой смене; на складе штат на пост учитывает коэффициент потерь рабочего времени. Остальной персонал процесса остаётся.
4. **Затраты.** Оборудование — цена из карточки. Внедрение, интеграция и зарядка — 15% от оборудования, если в карточке нет своей цифры. Обслуживание — 7% в год. Электроэнергия — мощность из карточки × часы × 7 ₽/кВт·ч; нет мощности — не учитывается, и это указано в предупреждении. Операторы парка — 1 на 10 роботов в смену. Резерв — 5%. Срок службы — из карточки или 7 лет.
5. **Сценарии.** «Как есть» — ФОТ замещаемых ролей за горизонт с ростом 9% в год. «Покупка» — CAPEX + OPEX + оставшиеся операторы. «RaaS» — 2,5% цены робота в месяц, сервис включён, внедрение оплачивается отдельно. Окупаемость — по накопленному денежному потоку без дисконтирования. Рекомендуется сценарий с наименьшим TCO, если он дешевле «как есть».
6. **Чувствительность.** ±20% к зарплате, цене оборудования, числу роботов, обслуживанию и энергии; показан срок окупаемости в оптимистичном и осторожном варианте.

Каждое допущение и каждое значение по умолчанию из паспорта перечисляются в разделе «Допущения» отчёта. Демо-расчёт остался только по ссылке </results/demo> (пример на лендинге).

Входные данные модели

- **Паспорт объекта** — значения полей `ParameterField`. У каждого поля есть единица измерения, допустимый диапазон, значение по умолчанию и источник норматива. Все они видны в форме и попадают в отчёт с пометкой «введено» или «по умолчанию».
- **Характеристики решения** — карточка каталога: цена, стоимость ПО, внедрения и обслуживания, производительность, автономность, срок службы. Поля, по которым приняты допущения, перечислены в `unconfirmedFields` и выводятся в отчёте.
- **Источники** — справочник `DataSource`, он же раздел «Источники» в отчёте.

Результат и воспроизводимость (3.1.5)

Результат модели — объект `CalculationResult` из контракта: KPI, сценарии, структура затрат, TCO, чувствительность, допущения, а также `dataVersion`, `modelVersion` и `calculatedAt`. Результат сохраняется в проект как снимок:

```
POST /api/projects/{id}/calculations
{ "calculation": { ..CalculationResult... }, "modelVersion": "economics-2026.09", "solutionId": "..." }
```

Паспорт объекта берётся из проекта и сохраняется в снимке вместе с результатом.

Каждый снимок хранится в истории проекта. При повторном открытии показывается сохранённый результат, а не пересчёт, — с указанием версий данных и модели.

Где пользователь видит формулы и допущения (3.5.8)

- Страница «Методика» (`/methodology`) — описание модели и значения по умолчанию.
- Страница результатов — допущения и структура затрат.
- Отчёт PDF — оговорка о предварительном характере оценки, параметры объекта с происхождением значений, выбранное решение с ограничениями, сценарии, CAPEX и OPEX, допущения, источники, версии данных и модели в колонтитуле.
- Excel — листы «Отчёт», «Сценарии», «CAPEX и OPEX», «Паспорт объекта», «Допущения», «Источники», «Чувствительность».

6. Загрузка и актуализация данных (ТЗ 6.6)

Что загружается при первом запуске

Сервис `migrate` применяет миграции и запускает `apps/api/prisma/seed.ts`. Сид берёт данные из `apps/api/prisma/seed-data`:

Файл	Содержимое
<code>object-types.json</code>	типы объектов: склад, аэропорт, медучреждение
<code>object-parameters.json</code>	поля паспортов с единицами, диапазонами, значениями по умолчанию и источниками. Значения по умолчанию взяты из демо-датасетов организатора
<code>robots-catalog/</code>	каталог роботов (v2): карточки по категориям, производители, коды, источники каждого значения (<code>sources.json</code>). Ведёт ответственный за каталог
<code>src/domain/catalogResearch.json</code>	характеристики из открытых источников (поиск через Perplexity 28.09.2026), каждое значение — со ссылкой на страницу. Принято только то, что автоматически сверено со страницей: число стоит на странице этой модели рядом с названием характеристики. Заполняет только пустые поля каталога, расхождения — в отчёт
<code>src/domain/catalogSupplements.json</code>	дополнения из материалов организатора: примеры решений по объектам, недостающие характеристики, решения вне каталога. Лежит в <code>src</code> , потому что нужен API и при загрузке версии из админки
<code>demo-calculation.json</code>	демонстрационный результат расчёта

Каталог для сیدا собирает скрипт `prisma/seed-data/build-catalog.ts` (запуск из `apps/api: npx tsx prisma/seed-data/build-catalog.ts [папка]`) в `src/domain/catalogSolutions.json`. Тот же файл использует демо-режим сайта. Правила сборки — `src/domain/robotCatalog.ts`, общие для скрипта, сیدا и загрузки из админки.

Сид применяет каталог так же, как загрузка новой версии: новые решения добавляются, характеристики обновляются из каталога, поля, правленные администратором, остаются как есть (`src/catalogSync.ts`).

Источники данных по умолчанию описаны в `apps/api/src/domain/sources.ts`.

Сид можно запускать повторно: он обновляет структуру полей (подпись, единицу, раздел, тип, варианты) и добавляет недостающие записи. Правки администратора он не перезаписывает: обязательность, значение по умолчанию, диапазон и источник норматива, отредактированные решения и источники остаются как есть.

Каталог решений

Новая версия каталога роботов (3.3.2, 3.3.6)

Ответственный за каталог присылает папку каталога (структура v2: `index.json`, `categories.json`, `vendors.json`, `codes.json`, `sources.json`, `data/*.json`).

- «Справочники» → «Каталог решений» → панель «Каталог роботов» → «Загрузить новую версию» → выберите папку. Отправляются только JSON-файлы; `schema/` и `README` не нужны.
- Сервер собирает каталог по тем же правилам, что и при первом запуске, с дополнениями организатора, и показывает отчёт без записи: сколько решений добавится, изменится (по каким полям), останется без изменений; какие решения из базы в новой версии отсутствуют; в каких сохраняются правки администратора; расхождения каталога с организатором; число решений по основаниям применимости для каждого объекта.
- «Применить версию» — запись одной транзакцией, событие в журнале изменений и новая версия данных.

Правила применения:

- характеристики и происхождение берутся из новой версии; поля, правленные администратором, не меняются;
- решения, которых нет в новой версии, **не удаляются** — на них могут ссылаться проекты. Удалить их можно вручную;
- фото хранятся на сайте (`apps/web/public/robots_photo`), а не в каталоге: у знакомых решений фото сохраняются, новым их добавляют в редакторе.
- Структура другой главной версии (не 2.x) отклоняется с понятной ошибкой.

Через API: `POST /api/admin/robot-catalog` (multipart, поле `files` — по файлу на поле, имя с путём внутри папки), `?dryRun=1` — только отчёт.

Таблица организатора (3.3.2)

- «Справочники» → «Каталог решений» → «Выгрузить xlsx». Получится таблица с актуальными колонками. Названия колонок совпадают с полями карточки: «Наименование», «Производитель», «Тип решения», «Цена, млн ₽», «Грузоподъёмность, кг»,

«Мин. ширина прохода, м» и так далее.

2. Перенесите данные организатора в эту таблицу. Распознаются и распространённые синонимы колонок: «название», «модель», «вендор», «стоимость», «цена с НДС», «страна», «навигация».
3. «Загрузить xls / csv» — выберите файл.
 - Строка с заполненным «ID» обновляет это решение.
 - Строка без «ID» ищет решение по паре «наименование + производитель». Если находит — обновляет, если нет — создаёт новое.
 - Строки с ошибками не сохраняются. После импорта показывается, сколько решений добавлено и обновлено, и список ошибок с номерами строк и причинами.
4. Импорт записывается в журнал изменений и меняет версию данных.

Через API: POST /api/catalog/solutions/import (multipart, поле file) и GET /api/catalog/solutions/export?format=xlsx|csv.

Дополнение из открытых источников (3.3.3, 3.3.4)

Происхождение ведётся по каждой характеристике (fieldSources в карточке решения). Генератор сворачивает факты robots-catalog/sources.json в поля карточки:

- источники значения — вид (материалы организатора, производитель, реселлер, СМИ, госреестр), название, ссылка, дата публикации или обращения;
- confirmed — значение в источнике точное. false — «около», значение серии, пересчёт или вывод; примечание каталога («паллет в час на зону, не на робота») сохраняется в note;
- составное поле (габариты) точное, только если точна каждая составляющая.

Допущения команды (страна по производителю) и дополнения организатора получают свою запись. В карточке каталога у значения стоят сноски на нумерованный список источников со ссылками, неточные значения помечены «приблизительно», «допущение» или «изменено вручную» с примечанием. В сравнении — та же метка.

Правка администратора (форма или импорт таблицы) заменяет происхождение изменённых полей записью «правка администратора»: прежний источник подтверждал прежнее значение. Повторный сид обновляет происхождение из каталога, но не трогает поля, правленные администратором.

Оценки команды для пустых полей

Последний слой сборки — src/domain/catalogEstimates.ts. Всё, что осталось пустым после каталога, материалов организатора и открытых источников, заполняется типовыми значениями класса оборудования: профиль категории (AMR, погрузчик, уборщик, БАС...) плюс уточнения по конкретным моделям, масса и габариты масштабируются по известной грузоподъёмности или массе. Экономика — доли цены единицы: ПО, обслуживание в год, внедрение на проект пилотного парка.

- Непустые значения не меняются: оценка заполняет только пропуски.
- У каждого оценённого поля в fieldSources источник «допущение команды» (kind: "team", confirmed: false) с примечанием, по какому классу оценено; в карточке — метка «допущение».
- Оценка не исключает решение из подбора: если оценённое значение не проходит проверку (проход, температура, лифты, допуск на перрон), решение получает «требуется проверки» с пометкой «по оценке команды».
- Оценённая производительность не задаёт размер парка: экономика считает его по скорости и маршруту объекта или по постам, как без неё. Строки затрат из оценок подписаны «оценка по классу оборудования» и требуют проверки.
- Реализованные кейсы не оцениваются: придуманный кейс — не допущение.

Ручная правка (3.3.5, 3.3.6)

«Справочники» → «Каталог решений» → «Добавить позицию» или «Редактировать» в строке решения. Форма содержит все группы полей таблицы 3.3 и показывает полноту карточки. Каталог обновляется по запросу администратора — правкой или импортом. Изменения видны пользователям сразу.

Паспорта объектов и нормативы

- «Справочники» → «Нормативы и значения по умолчанию»: для каждого поля задаются значение по умолчанию, минимум и максимум, источник и обязательность. Значение по умолчанию проверяется на попадание в диапазон.
- Новые поля и типы объектов добавляются в object-parameters.json и object-types.json, после чего сид перезапускается: docker compose run --rm migrate.

Параметры объекта пользователя (3.2.3)

На шаге паспорта нажмите «Скачать шаблон Excel». В шаблоне колонки «ID поля», «Раздел», «Параметр», «Значение», «Ед. изм.», «Обязательное», «Допустимые значения», «По умолчанию», «Источник норматива», «Подсказка». Заполните колонку «Значение» и загрузите файл кнопкой «Загрузить из Excel / CSV».

Строки сопоставляются по «ID поля» или по названию параметра. Для каждого значения проверяются тип, единица измерения, диапазон и допустимые варианты. После загрузки показывается, сколько полей распознано, какие строки пропущены и какие ошибки найдены, с указанием строки и способа исправления.

Через API: GET `/api/object-types/{slug}/parameters/template?format=xlsx|csv` и POST `/api/object-types/{slug}/parameters/import`.

Версии данных

Каждая правка каталога, нормативов или источников пишется в журнал изменений: пользователь, время, что изменилось, значения до и после. Отметка последней правки — это текущая версия данных. Она сохраняется в каждом снимке расчёта, поэтому видно, на каких данных сделан расчёт.

Резервное копирование

```
docker compose exec db pg_dump -U robopodbor robopodbor > backup.sql
docker compose exec -T db psql -U robopodbor robopodbor < backup.sql
```

7. Руководство пользователя (ТЗ 6.7)

Без входа

- **Каталог решений** (/catalog) — сетка карточек роботов с фото (или плейшолдером категории, если фото нет). Слева дерево «объект → процесс → категория каталога» и ветка «Весь каталог по категориям» (29 категорий, в том числе роботы вне склада, аэропорта и клиники). Подробные фильтры — зрелость, страна, цена; фильтры «Доступность» и «Данные» появляются, когда в каталоге есть что различать. Поиск ищет по названию, производителю, назначению, категории и ID, не зависит от регистра, «ё», дефисов и пробелов и понимает латиницу и кириллицу: «rubi» найдёт «РУБИ-С-03», «ронави» — «Ronavi H1500». Отметьте несколько решений на карточках и нажмите «Сравнить».
- **Сравнение** (/catalog/compare) — характеристики сгруппированы как в таблице 3.3 ТЗ. Значения, принятые как допущение, помечены. Если выбран объект, в сравнении есть строка подбора: подходит решение или нет и почему.
- **Расчёт** — кнопка «Рассчитать» на главной. Гость проходит мастер и получает полный расчёт; ссылка на отчёт открывается 90 дней. Сохранить проект можно только после входа. Мастер запоминает введённое: после перезагрузки страницы параметры и выбранный робот остаются.

Мастер расчёта

1. **Тип объекта** — склад, аэропорт или медучреждение.
2. **Параметры** — паспорт объекта, разбитый по разделам.
 - У каждого поля указаны единица измерения, подсказка, значение по умолчанию и источник норматива. Обязательные поля отмечены.
 - Пустое поле берётся по умолчанию.
 - Ошибки показываются у поля с допустимым диапазоном и тем, как исправить значение.
 - «Скачать шаблон Excel» — таблица со всеми полями. Заполните колонку «Значение» и нажмите «Загрузить из Excel / CSV». Платформа покажет, сколько полей распознано и какие строки не приняты и почему.
3. **Процессы** — выбор процессов и решения.
 - Над списком сводка подбора: сколько решений подходит, сколько требует проверки, сколько не подходит.
 - У каждого решения показаны балл и вклад критериев, причины соответствия, ограничения и недостающие данные.
 - Неподходящие решения собраны в свёрнутый список «Не подходят для объекта» с причиной исключения. Их можно выбрать вручную — появится предупреждение.
4. **Расчёт** — экономика считается для выбранного робота по паспорту объекта. Без выбранного робота расчёт не запускается.
5. **Результат** — KPI, сравнение сценариев «как есть», «покупка» и «услуга», структура CAPEX и OPEX, чувствительность, допущения и 3D-симуляция склада. Над отчётом — предупреждение, каких данных не хватило. В «Допущениях» перечислены все принятые допущения и поля паспорта, взятые по умолчанию. Если у робота в карточке нет скорости, времени работы, зарядки или мощности, на сцене отмечено, что взято у демо-робота. Кнопки **PDF** и **Excel** выгружают отчёт и расчётные таблицы.

Отчёт

PDF начинается с оговорки, что это предварительная оценка, требующая проверки при обследовании объекта. Дальше в отчёте:

- KPI;
- выбранное решение со статусом подбора, ограничениями и недостающими данными;
- сценарии;
- структура затрат;
- параметры объекта с пометкой «введено» или «по умолчанию» и источником;
- допущения;
- источники данных.

В колонтитуле каждой страницы — дата расчёта, версия данных и версия модели. Excel содержит те же разделы на отдельных листах.

Личный кабинет (после входа)

Меню кабинета: «Обзор», «Мои расчёты», «Избранные роботы», «Каталог решений», «Учётная запись». То же меню — под аватаром в шапке.

- **Обзор** — только ваши данные: сколько расчётов сохранено (с результатом и черновиков), сколько роботов в избранном, самая быстрая окупаемость и последний расчёт; ниже последние расчёты, избранные роботы и быстрые действия. Администратор дополнительно видит последние правки каталога.
- **Избранные роботы** — роботы, отмеченные ♥ на карточке в каталоге или в карточке робота. Отметьте 2–4 и нажмите «Сравнить выбранные» — характеристики построчно. «Карточка» открывает робота в каталоге. Избранное хранится в аккаунте

и одинаково на всех устройствах.

- **Учётная запись** — идентификационные данные (ФИО, почта-логин, организация, роль, дата регистрации, ID), правка ФИО и организации, смена пароля (нужен текущий; после смены другие устройства выходят из аккаунта), выход.

Мои расчёты (после входа)

- **Сохранить** — кнопка на экране результатов, рядом с PDF и XLS: расчёт попадает в «Мои расчёты». Гостя кнопка ведёт на вход и возвращает к тому же результату.
- **Сравнить** — отметьте галочкой 2–4 расчёта с результатом и нажмите «Сравнить»: окупаемость, CAPEX, ROI, снижение OPEX, TCO по сценариям, роботы и дата расчёта бок о бок; лучшее значение в строке подсвечено.
- **Сохранить черновик** — после прохождения мастера сохраняет проект вместе с параметрами, выбранным решением и расчётом (если робот выбран).
- В меню строки проекта:
 - **История расчётов** — все сохранённые расчёты с датой, версией данных, версией модели, решением и окупаемостью. «Открыть» показывает расчёт ровно таким, каким он был сохранён.
 - **Переименовать** — название и статус: черновик, на согласовании, защищён.
 - **Скопировать** — копия со всеми параметрами и расчётом.
 - **Удалить** — удаляет проект и всю историю расчётов после подтверждения.
- Чужие проекты недоступны: каждый пользователь видит только свои.

Типичные сообщения

Сообщение	Что сделать
«Значение меньше допустимого минимума ...»	Ввести число в указанном диапазоне или очистить поле, чтобы взять значение по умолчанию
«Не распознано ни одно поле»	Скачать шаблон и заполнить колонку «Значение», не меняя заголовки
«Файл не читается»	Сохранить файл как .xlsx или .csv в UTF-8
«Требуется проверки» у решения	Посмотреть «Недостающие данные» и уточнить их у поставщика или при обследовании

8. Руководство администратора (ТЗ 6.7, 3.1.4, 3.8)

Вход: `admin@robotopodbor.ru / admin123`. В боковом меню появляется блок «Администрирование»: «Редактор каталога» (`/admin/catalog`) и «Справочники и журнал» (`/admin`). Пользователям без роли администратора он не показывается, а API отвечает им 403.

Справочники

Источники данных

Список источников, на которые ссылаются каталог, нормативы и отчёт. У источника есть название, область применения, тип (цены, тарифы, реестр, бенчмарк, внедрения, датасет, открытые источники), ссылка, дата актуализации и признак «подтверждён».

- «Добавить источник» — новая запись.
- Карандаш в строке — изменить запись. Корзина — удалить после подтверждения.

Источники выводятся в разделе «Источники» отчёта PDF и на листе Excel.

Нормативы и значения по умолчанию

Выберите тип объекта. Для каждого поля паспорта можно задать:

- значение по умолчанию — оно подставляется, если пользователь не заполнил поле;
- минимум и максимум;
- источник норматива — показывается пользователю под полем и в отчёте;
- обязательность.

Сохранение проверяет, что минимум не больше максимума и что значение по умолчанию попадает в диапазон. Если нет, показывается ошибка с причиной.

Журнал изменений

Все правки каталога, нормативов и источников, а также импорты: время, пользователь, действие, краткое описание и таблица «было → стало». Журнал можно отфильтровать по типу записи. Последняя запись определяет текущую версию данных, которая сохраняется в новых расчётах.

Каталог решений

«Редактор каталога» в меню «Администрирование» или под аватаром в шапке (`/admin/catalog`). Поля, заполненные оценкой команды, помечены «допущение»; правка администратора заменяет оценку и помечается «изменено вручную».

- **Таблица** — все решения с типом, ценой, полнотой карточки и подтверждённостью данных. Справа — дерево «объект → процесс → тип решения».
- **Добавить позицию или «Редактировать»** — форма со всеми группами полей таблицы 3.3 ТЗ:
 - идентификация;
 - технические характеристики;
 - ограничения для подбора: ширина прохода, ровность пола, высота, температура, лифты, допуск на перрон;
 - инфраструктура;
 - экономика;
 - применимость;
 - качество данных.

Список процессов зависит от выбранных типов объектов. Полнота карточки пересчитывается при вводе. Обязательные поля и числа проверяются до сохранения.

Поля каталога роботов:

- **Категория каталога** — код из 29 категорий (FC, AM, FL...), по ней роботы делятся на подкатегории при выборе.
- **УГТ** — уровень готовности технологии, 1–9.
- **Типы объектов и основание** — для каждого объекта: «не подходит», «заявлено в каталоге», «пример организатора» или «выведено по сценариям». «Выведено» понижает статус подбора до «Требуется проверки».
- **Фото** — имена файлов из `apps/web/public/robots_photo`, первое — главное. После добавления файлов запустите `bash scripts/build-robot-previews.sh` в `apps/web`.

Изменённые поля получают источник «правка администратора» — в карточке они помечены «изменено вручную». Повторный сид не перезаписывает правленные характеристики, применимость, процессы и фото.

- **Каталог роботов** — панель над таблицей: текущая версия каталога и загрузка новой версии папкой с предпросмотром изменений. Порядок и правила — в [06-data.md](#).
- **Удалить** — после подтверждения.
- **Загрузить xlsx / csv, Выгрузить xlsx, CSV** — работа с таблицей решений, порядок описан в [06-data.md](#). В таблице есть колонки «Категория каталога», «УГТ», «Фото» и «Основание применимости» в виде `Склад: заявлено в каталоге; Аэропорт: пример организатора` — выгрузка и обратная загрузка их не теряют.

Изменения видны пользователям сразу: кеш справочников сбрасывается при каждой правке.

Учётные записи

Пользователи регистрируются сами на странице входа и получают роль `user`. Роль администратора назначается в базе:

```
docker compose exec db psql -U robopodbor robopodbor \  
-c "UPDATE \"User\" SET role='admin' WHERE email='name@example.ru';"
```

9. Внешние библиотеки и источники данных (ТЗ 6.8)

Все библиотеки — свободное ПО с открытым исходным кодом. Точные версии закреплены в `apps/web/package-lock.json` и `apps/api/package-lock.json`.

Сервер (`apps/api`)

Библиотека	Назначение	Лицензия	Ссылка
Node.js 22	среда выполнения	MIT	https://nodejs.org
Express 4	HTTP-сервер	MIT	https://expressjs.com
Prisma 6	ORM и миграции	Apache-2.0	https://www.prisma.io
zod 3	проверка входных данных	MIT	https://zod.dev
multer 2	приём файлов	MIT	https://github.com/expressjs/multer
helmet 8	заголовки безопасности	MIT	https://helmetjs.github.io
cors, cookie-parser	CORS и cookie	MIT	https://github.com/expressjs
SheetJS CE 0.20	чтение и запись <code>xlsx/csv</code>	Apache-2.0	https://sheetjs.com
tsx, TypeScript	сборка и запуск	MIT, Apache-2.0	https://www.typescriptlang.org

Интерфейс (`apps/web`)

Библиотека	Назначение	Лицензия	Ссылка
React 19	интерфейс	MIT	https://react.dev
Vite	сборка	MIT	https://vite.dev
React Router 6	маршрутизация	MIT	https://reactrouter.com
TanStack Query 5	загрузка и кеш данных	MIT	https://tanstack.com/query
Zustand 5	состояние мастера	MIT	https://zustand.docs.pmnd.rs
openapi-fetch, openapi-typescript	типизированный клиент по контракту	MIT	https://openapi-ts.dev
react-hook-form	формы	MIT	https://react-hook-form.com
Radix UI	доступные примитивы	MIT	https://www.radix-ui.com
Tailwind CSS	стили	MIT	https://tailwindcss.com
Recharts	графики	MIT	https://recharts.org
three.js, @react-three/fiber, drei	3D-сцена и визуализация	MIT	https://threejs.org
jsPDF, jspdf-autotable	отчёт PDF	MIT	https://github.com/parallax/jsPDF
SheetJS CE	выгрузка Excel	Apache-2.0	https://sheetjs.com
lucide-react	иконки	ISC	https://lucide.dev
MSW	моки API для демо без сервера и тестов	MIT	https://mswjs.io
Vitest, Testing Library	тесты	MIT	https://vitest.dev

Шрифты:

- PT Sans — для отчёта PDF, лицензия SIL OFL 1.1, текст лицензии в `apps/web/src/assets/fonts/PTSans-0FL.txt`. Ссылка: <https://www.paratype.ru/public/>.
- Inter, Space Grotesk, JetBrains Mono — для интерфейса, лицензия SIL OFL 1.1, пакеты Fontsource.

Инфраструктура

Компонент	Лицензия	Ссылка
PostgreSQL 16	PostgreSQL License	https://www.postgresql.org
nginx	BSD-2-Clause	https://nginx.org
Caddy 2	Apache-2.0	https://caddyserver.com
Docker, Docker Compose	Apache-2.0	https://www.docker.com

Источники данных

Актуальный список ведёт администратор в разделе «Справочники» → «Источники данных». Он же выводится в каждом отчёте. Начальный набор:

Источник	Что взято	Актуальность
Каталог роботов (robots_catalog v2.0.0)	характеристики, категории и применимость 187 роботов	25.09.2026
Каталог решений ФЦ БАС, выгрузка catalog_export_v4.csv	цены изделий	21.09.2026
Демо-датасеты объектов «Датасеты_хакатон.xlsx»	паспорта склада, аэропорта и медучреждения, значения по умолчанию	2026
Материалы организатора: ТЗ и «Дополнения для участников»	примеры решений для объектов, недостающие характеристики	09.2026
Сайты производителей и дистрибьюторов	недостающие характеристики, сверенные со страницей модели	09.2026
Оценки модели экономики	внедрение 15% и обслуживание 7% цены, RaaS 2,5% цены в месяц, электроэнергия 7 ₺/кВт·ч	09.2026, не подтверждено

У каждого решения каталога хранятся собственные источник, ссылка, дата актуализации и признак подтверждённости (ТЗ 3.3.4).

10. Известные ограничения и план развития (ТЗ 6.9, 5.7)

Что реализовано и в каком объёме

Функция	Статус
Роли гость, пользователь, администратор; разграничение доступа	полностью
Проекты: создание, переименование, смена статуса, копирование, удаление, история расчётов с версиями	полностью
Паспорта трёх типов объектов, ручной ввод, шаблон и импорт Excel/CSV с проверкой	полностью
Каталог: иерархия, фильтры, поиск, сортировка, сравнение, качество данных	полностью
Подбор с исключениями, причинами, «требуется проверки» и объяснимым баллом	полностью
Администрирование: каталог, импорт и экспорт таблицы, источники, нормативы, журнал изменений	полностью
Отчёт PDF и Excel с оговоркой, допущениями, источниками и версиями	полностью
Экономическая модель и сценарии	реализовано как экспресс-оценка (05-economics.md), упрощения перечислены ниже
Визуализация работы роботов	только склад: 3D-сцена с пылесосами, манипуляторами и погрузчиками
Роль вендора, проверка по госреестрам, меры господдержки	не реализовано (дополнительные функции)
Интеграции с WMS, ERP, 1C	описаны, не реализованы (11-integrations.md)

Известные ограничения

- Данные каталога.** Каталог роботов (v2.0.0, 187 роботов) заполнен неравномерно. У складских роботов, которые показывает симуляция, чаще всего нет ёмкости батареи или мощности (27 из 34), времени зарядки (18) и времени работы (12): производители их не публикуют. Недостающее симуляция берёт у демо-робота того же типа и перечисляет это у сцены; расчёт экономики не учитывает электроэнергию и помечает это в предупреждении. Фото есть у 31 робота из 74, участвующих в подборе.
- Упрощения экономики.** Нет дисконтирования и налогов. Робот замещает один пост в каждой смене. Внедрение, обслуживание, аренда и тариф на электроэнергию (один на все регионы) — оценки. Замещаемые роли заданы по процессу: для уборки склада, охраны, инвентаризации, обслуживания пассажиров и пожарной безопасности в паспорте нет персонала, и окупаемость не считается.
- Симуляция только складская.** Для аэропорта и клиники 3D-сцены нет.
- Сайт на GitHub Pages без сервера.** Если переменная `API_URL` не задана, сайт работает на встроенных моках: аккаунты, проекты и расчёты хранятся в `localStorage` браузера и не видны с другого устройства. Как подключить настоящий сервер — в [02-deploy.md](#).
- Удалённые решения каталога** возвращаются при повторном запуске сида (`docker compose up` запускает его каждый раз): сид загружает каталог роботов целиком. Правки администратора в существующих решениях сид не трогает.
- Роль администратора** назначается запросом к базе, отдельного интерфейса управления пользователями нет.
- Автоматическое обновление каталога** не реализовано. Обновление выполняется по запросу администратора: вручную или импортом таблицы.
- Весы ранжирования** — экспертное допущение команды, они не калибровались на реальных внедрениях.
- Проверки подбора** покрывают ключевые ограничения: груз, проходы, пол, высота, температура, перрон, лифты, бюджет. Более тонкие условия (радиус разворота, уклоны, пропускная способность лифтов) не проверяются и попадают в «Недостающие данные» или в ограничения карточки.
- Файлы** разбираются в памяти и не хранятся. Исходный файл импорта повторно скачать нельзя, сохраняются только распознанные значения.
- Масштаб.** Нагрузочный тест на 50 одновременных пользователей (ТЗ 4.3.1) проведён на одном экземпляре API без nginx, результаты — в [03-architecture.md](#). На сервере с другим железом их стоит повторить тем же скриптом. Кеш справочников локален для каждого экземпляра API: при нескольких экземплярах правка видна остальным не позже чем через 60 секунд.

План развития

- Получить от ответственного за каталог недостающие характеристики (батарея, зарядка, производительность) и фото, закрыть расхождения, найденные при сверке с открытыми источниками.
- Расширить экономику: роли персонала для всех процессов, региональные тарифы, дисконтирование для защиты бюджета.
- Добавить плановое обновление каталога из открытых источников с модерацией изменений через журнал.
- Добавить роль вендора: поставщик ведёт карточки своих решений, администратор подтверждает изменения.

5. Проверять российское происхождение по реестру ГИСП через API и учитывать меры господдержки в расчёте.
6. Добавить интеграционный API для WMS, ERP и 1С по [11-integrations.md](#).
7. Калибровать веса ранжирования по данным реализованных кейсов.
8. Добавить 3D-сцены аэропорта и клиники.
9. Добавить новые типы объектов (производство, ритейл, агро) данными, без изменения кода.

11. Возможные интеграции (ТЗ 3.8)

Сейчас данные поступают из файлов (Excel/CSV) и встроенной базы. Все операции импорта уже доступны как API (ТЗ 3.8.2), поэтому внешняя система подключается к тем же точкам, что и интерфейс. Описание — в [contracts/openapi.yaml](#).

Данные	Точка входа
Параметры объекта	POST /api/object-types/{slug}/parameters/import (файл) или POST /api/projects с полем parameters
Каталог решений	POST /api/catalog/solutions/import, POST/PUT /api/catalog/solutions
Нормативы	PATCH /api/admin/parameters/{fieldId}
Источники данных	POST/PUT /api/admin/sources
Расчёт по паспорту и решению	POST /api/calculations
Результат внешней модели	POST /api/projects/{id}/calculations

Для интеграций «сервер — сервер» потребуется отдельный механизм доступа (сервисные токены вместо cookie-сессии). Это первый шаг в плане развития.

WMS (система управления складом)

- **Что получаем:** фактические объёмы приёмки, отбора и отгрузки по часам, число SKU, структуру ячеек, длину маршрутов. Это точнее ручного ввода паспорта и даёт реальную пиковую нагрузку для расчёта количества роботов.
- **Как:** выгрузка агрегатов из WMS за период, преобразование в поля паспорта wh_*, создание проекта через API.
- **Обратно:** требования к WMS со стороны выбранного решения (постановка задач роботам, интерфейс систем управления парком) — из карточки решения, раздел «Интеграция».

ERP

- **Что получаем:** фонд оплаты труда по подразделениям, стоимость смены, амортизационные политики, ставку дисконтирования компании. Это замена отраслевых оценок в допущениях расчёта.
- **Обратно:** CAPEX и OPEX по годам в разрезе статей из Excel-выгрузки для формирования инвестиционной заявки.

1C

- **1C:ЗУП** — численность и стоимость персонала по сменам.
- **1C:Бухгалтерия / 1C:ERP** — тарифы на электроэнергию, фактические расходы на обслуживание текущего оборудования.
- **Как:** регламентная выгрузка в Excel по шаблону паспорта или HTTP-сервис 1C, вызывающий API платформы.

Системы мониторинга оборудования

- **Что получаем после внедрения:** фактическую производительность, доступность, простои и зарядку роботов.
- **Зачем:** сравнить факт с расчётом, подтвердить или скорректировать характеристики в каталоге (признак «подтверждено») и калибровать веса подбора.
- **Как:** периодическая передача агрегатов (телеметрия по MQTT или REST у поставщика → адаптер → обновление карточки решения с записью в журнал изменений).

Аналитические продукты ФЦ БАС

- **Каталог:** обмен таблицей решений в формате выгрузки платформы (GET /api/catalog/solutions/export) — те же колонки, что и при импорте.
- **Аналитика спроса:** обезличенная статистика проектов — типы объектов, выбранные решения, диапазоны окупаемости. Для этого нужен отдельный агрегирующий API без доступа к проектам пользователей.
- **Госреестры:** проверка российского происхождения по реестру ГИСП и учёт мер поддержки — дополнительная функция 3.8.4, источник уже заведён в справочнике.